

Условия и циклы

Полное обучение на примере Python 3: if/elif/else, match, циклы for и while, break, continue, else у циклов, range, enumerate, zip, вложенные циклы и генераторы списков.

1. Зачем нужны условия и циклы

Без них программа выполняется строго сверху вниз, строка за строкой. **Условия** позволяют выбирать, какой код выполнить, а **циклы** — повторять код много раз.

Конструкция	Что делает	Пример из игр
<code>if</code>	выполняет код, только если условие истинно	если HP ≤ 0 — игрок погиб
<code>if / else</code>	выбирает одну из двух веток	попал или промахнулся
<code>if / elif / else</code>	выбирает одну из многих веток	редкость лута
<code>for</code>	повторяет для каждого элемента	обойти всех врагов
<code>while</code>	повторяет, пока условие истинно	игровой цикл, пока игрок жив

2. Отступы — основа синтаксиса

В Python блок кода задаётся **отступом** (стандарт — 4 пробела), а не фигурными скобками, как в C# или JS. Строка с условием или циклом заканчивается **двоеточием**.

```
python
hp = 0
if hp <= 0:
    print("Игрок погиб")      # внутри if (есть отступ)
    print("Game Over")       # тоже внутри if
print("Эта строка выполнится всегда") # вне if (без отступа)
```



Смешивать табы и пробелы нельзя — будет `IndentationError` или `TabError`. Настрой редактор так, чтобы Tab вставлял 4 пробела.

3. Условный оператор if

3.1. Простой if

```
python

gold = 120
if gold >= 100:
    print("Можно купить меч")
```

3.2. if / else

```
python

roll = 7
if roll >= 10:
    print("Попадание!")
else:
    print("Промаш")
```

3.3. if / elif / else

Проверки идут **сверху вниз**, выполняется **только первая** подходящая ветка. Остальные пропускаются.

```
python

score = 85
if score >= 90:
    grade = "S"
elif score >= 75:
    grade = "A"      # выполнится эта ветка
elif score >= 50:
    grade = "B"
else:
    grade = "C"
print(grade)        # A
```



Порядок важен. Если поставить `score >= 50` первым, то при 85 сработает именно он, и оценка будет `B`. Сначала проверяй самые строгие условия.

3.4. Сложные условия: and, or, not

```
python

has_key = True
level = 12
if has_key and level >= 10:
    print("Дверь в данж открыта")

is_night = False
has_torch = True
if not is_night or has_torch:
    print("Видно дорогу")

hp = 45
if 0 < hp <= 50:      # цепочка сравнений
    print("Мало здоровья")
```

3.5. Вложенные условия

```
python

is_alive = True
mana = 30
if is_alive:
    if mana >= 25:
        print("Кастую огненный шар")
    else:
        print("Не хватает маны")
```

Глубокую вложенность лучше заменять на `and` или «ранний выход» (см. раздел про функции позже):

```
python

if is_alive and mana >= 25:
    print("Кастую огненный шар")
```

3.6. Проверка на «пустоту» (truthy / falsy)

```
python

inventory = []
if inventory:
    print("Есть предметы")
else:
    print("Инвентарь пуст")    # пустой список – это False

name = ""
if not name:
    name = "Безымянный"
```



Ложными считаются: `False`, `0`, `0.0`, `""`, `None`, `[]`, `{}`, `()`, `set()`. Всё остальное — ИСТИННО.

3.7. Тернарный оператор (if в одну строку)

```
python

hp = 0
status = "мёртв" if hp <= 0 else "жив"
print(status)    # мёртв
```

Используй его только для простых случаев. Длинную логику пиши обычным `if`.

3.8. match / case (Python 3.10+)

Удобен, когда нужно сравнить одно значение со многими вариантами.

```
python

command = "attack"
match command:
    case "attack":
        print("Атакую!")
    case "defend" | "block":    # несколько вариантов через |
        print("Защищаюсь")
    case "run":
        print("Убегаю")
    case _:                    # _ — «всё остальное», как else
        print("Неизвестная команда")
```

`match` умеет и больше — разбирать структуру данных:

```
python

event = ("damage", 25)
match event:
    case ("damage", amount):
        print(f"Получен урон: {amount}")
    case ("heal", amount):
        print(f"Лечение: {amount}")
```

4. Цикл for

`for` перебирает элементы **любой коллекции** по очереди: список, строку, словарь, диапазон чисел.

4.1. Перебор списка

```
python

enemies = ["гоблин", "орк", "тролль"]
for enemy in enemies:
    print(f"Атакую: {enemy}")
```

4.2. range — диапазон чисел

Вызов	Числа	Пояснение
<code>range(5)</code>	0, 1, 2, 3, 4	от 0 до 5, не включая 5
<code>range(2, 6)</code>	2, 3, 4, 5	от 2 до 6, не включая 6
<code>range(0, 10, 3)</code>	0, 3, 6, 9	шаг 3
<code>range(5, 0, -1)</code>	5, 4, 3, 2, 1	обратный отсчёт

```
python

for i in range(3):
    print("Волна", i + 1)  # Волна 1, Волна 2, Волна 3

for t in range(3, 0, -1):
    print(t)               # 3, 2, 1
print("Старт!")
```

 Конец диапазона **не включается**: `range(1, 10)` даёт числа от 1 до 9. Чтобы включить 10, пиши `range(1, 11)`.

4.3. Перебор строки

```
python

for ch in "HP":
    print(ch)  # H, затем P
```

4.4. enumerate — индекс и значение сразу

```
python

items = ["меч", "щит", "зелье"]
for i, item in enumerate(items, start=1):
    print(f"{i}. {item}")

# 1. меч
# 2. щит
# 3. зелье
```



Вместо `for i in range(len(items))` почти всегда лучше `enumerate(items)` — короче и без ошибок с индексами.

4.5. Перебор словаря

```
python

stats = {"hp": 100, "mana": 40, "speed": 7}

for key in stats:                # ТОЛЬКО КЛЮЧИ
    print(key)

for key, value in stats.items(): # КЛЮЧИ И ЗНАЧЕНИЯ
    print(f"{key} = {value}")

total = 0
for value in stats.values():     # ТОЛЬКО ЗНАЧЕНИЯ
    total += value
print(total)                    # 147
```

4.6. zip — несколько списков параллельно

```
python

names = ["Arsi", "Kai", "Mira"]
scores = [120, 95, 140]
for name, score in zip(names, scores):
    print(f"{name}: {score}")
```

`zip` останавливается на самом коротком списке.

4.7. Накопление результата

Типичные шаблоны: сумма, счётчик, поиск максимума, фильтрация.

```
python

damage_log = [12, 30, 7, 45, 18]

total = 0
for d in damage_log:
    total += d
print(total)          # 112

crits = 0
for d in damage_log:
    if d >= 30:
        crits += 1
print(crits)          # 2

best = damage_log[0]
for d in damage_log:
    if d > best:
        best = d
print(best)           # 45 (то же самое: max(damage_log))

big_hits = []
for d in damage_log:
    if d > 15:
        big_hits.append(d)
print(big_hits)       # [30, 45, 18]
```

5. Цикл while

while повторяет код, **пока условие истинно**. Используйте его, когда заранее неизвестно, сколько будет повторений.

```
python

hp = 100
turn = 0
while hp > 0:
    turn += 1
    hp -= 30
    print(f"Ход {turn}: HP = {hp}")
print("Игрок погиб")
# Ход 1: HP = 70
# Ход 2: HP = 40
# Ход 3: HP = 10
# Ход 4: HP = -20
```



Бесконечный цикл. Если условие никогда не станет ложным, программа зависнет. Внутри `while` обязательно должно меняться то, что проверяется в условии. Остановить зависшую программу в терминале — `Ctrl + C`.

```
python

# ОШИБКА: hp не меняется — цикл никогда не закончится
# while hp > 0:
#     print("бой идёт")
```

5.1. while True + break

Частый шаблон для игровых циклов и меню: крутимся бесконечно и выходим через `break`.

```
python

while True:
    answer = input("Продолжить? (да/нет): ")
    if answer == "нет":
        break
    print("Играем дальше!")
print("Выход из игры")
```

5.2. for или while?

Ситуация	Что выбрать
Перебрать все элементы коллекции	<code>for</code>
Повторить ровно N раз	<code>for i in range(N)</code>
Повторять, пока что-то не случится	<code>while</code>
Ждать правильного ввода пользователя	<code>while True</code> + <code>break</code>
Игровой цикл «пока игра не закончена»	<code>while</code>

6. Управление циклом: break, continue, else

6.1. break — досрочный выход

```
python

loot = ["камень", "палка", "алмаз", "кость"]
for item in loot:
    if item == "алмаз":
        print("Нашёл алмаз!")
        break          # дальше не ищем
    print(f"{item} — не то")
```

6.2. continue — пропустить итерацию

```
python

for n in range(1, 8):
    if n % 2 == 0:
        continue      # чётные пропускаем
    print(n)           # 1, 3, 5, 7
```

6.3. else у цикла

Блок `else` после цикла выполняется, **если цикл завершился сам**, без `break`. Удобно для поиска.

```
python

enemies = ["гоблин", "орк"]
for e in enemies:
    if e == "дракон":
        print("Дракон найден!")
        break
else:
    print("Дракона нет")    # выполнится: break не сработал
```

6.4. pass — пустой блок

Python не разрешает пустые блоки. `pass` — заглушка «ничего не делать».

```
python

for i in range(3):
    pass          # TODO: добавить логику позже
```

7. Вложенные циклы

Цикл внутри цикла: внутренний проходит полностью **на каждой** итерации внешнего.

```
python

for row in range(3):
    line = ""
    for col in range(4):
        line += "#"
    print(line)
# ####
# ####
# ####
```

Обход игровой сетки (карты):

```
python

grid = [
    [0, 1, 0],
    [0, 0, 1],
    [1, 0, 0],
]
walls = 0
for y, row in enumerate(grid):
    for x, cell in enumerate(row):
        if cell == 1:
            walls += 1
            print(f"Стена в ({x}, {y})")
print("Всего стен:", walls)  # 3
```



break выходит только из **ближайшего** цикла. Чтобы выйти из обоих, используй флаг-переменную или вынеси циклы в функцию и сделай **return**.

```
python

found = False
for y in range(3):
    for x in range(3):
        if grid[y][x] == 1:
            found = True
            break
    if found:
        break
print(x, y)  # 1 0 — первая стена
```

8. Генераторы списков (list comprehension)

Короткая запись «создай список с помощью цикла».

```
python

# Обычный цикл
squares = []
for n in range(5):
    squares.append(n ** 2)

# То же самое одной строкой
squares = [n ** 2 for n in range(5)]
print(squares)          # [0, 1, 4, 9, 16]

# С условием-фильтром
evens = [n for n in range(10) if n % 2 == 0]
print(evens)            # [0, 2, 4, 6, 8]

# С преобразованием строк
names = ["гоблин", "орк"]
upper = [n.upper() for n in names]
print(upper)            # ['ГОБЛИН', 'ОРК']

# Генератор словаря
hp_map = {name: 100 for name in names}
print(hp_map)           # {'гоблин': 100, 'орк': 100}
```



Генераторы хороши для простых преобразований. Если внутри больше одного условия или сложная логика — пиши обычный цикл, так понятнее.

Полезные встроенные функции, которые заменяют простые циклы:

Функция	Что делает	Пример	Результат
<code>sum()</code>	сумма	<code>sum([1, 2, 3])</code>	6
<code>max()</code> / <code>min()</code>	максимум / минимум	<code>max([4, 9, 2])</code>	9
<code>len()</code>	количество	<code>len("hello")</code>	5
<code>any()</code>	хотя бы один True	<code>any([0, 0, 5])</code>	True
<code>all()</code>	все True	<code>all([1, 2, 0])</code>	False
<code>sorted()</code>	отсортированная копия	<code>sorted([3, 1, 2])</code>	[1, 2, 3]

9. Частые ошибки новичков

Ошибка	Что происходит	Как правильно
нет <code>:</code> после <code>if x > 0</code>	SyntaxError	<code>if x > 0:</code>
<code>if x = 5:</code>	SyntaxError	<code>if x == 5:</code>
забыли отступ внутри блока	IndentationError	4 пробела внутри блока
<code>range(1, 10)</code> ждут 10	10 не входит	<code>range(1, 11)</code>
<code>while</code> без изменения условия	бесконечный цикл	менять переменную внутри
<code>if x == 1 or 2:</code>	всегда True	<code>if x == 1 or x == 2:</code> или <code>x in (1, 2)</code>
удаление из списка во время <code>for</code>	элементы пропускаются	перебирать копию: <code>for e in enemies[:]</code>
<code>else</code> у цикла считают «если пусто»	срабатывает, когда нет <code>break</code>	помнить правило из 6.3

Разбор ошибки `if x == 1 or 2`: Python читает это как `(x == 1) or 2`, а число `2` всегда истинно — поэтому условие выполняется всегда.

```
python

x = 7
if x == 1 or 2:
    print("Сработало!")    # выведет, хотя x = 7

if x in (1, 2):
    print("Не выведет")    # правильно
```

10. Мини-пример: всё вместе

Маленькая пошаговая битва с волнами врагов.

```
python

import random

random.seed(42) # одинаковый результат при каждом запуске
player_hp = 100
waves = [["гоблин", "гоблин"], ["орк"], ["тролль", "гоблин"]]
enemy_hp = {"гоблин": 20, "орк": 45, "тролль": 70}
kills = 0

for wave_num, wave in enumerate(waves, start=1):
    print(f"=== Волна {wave_num} ===")
    for enemy in wave:
        hp = enemy_hp[enemy]
        while hp > 0 and player_hp > 0:
            hit = random.randint(10, 25)
            crit = random.random() < 0.2
            if crit:
                hit *= 2
            hp -= hit
            print(f"    Бью {enemy}: -{hit}{' КРИТ!' if crit else ''}")
            if hp > 0:
                player_hp -= 5
            if player_hp <= 0:
                break
            kills += 1
            print(f"    {enemy} повержен. HP игрока: {player_hp}")
        else:
            continue # волна пройдена без break — идём к следующей
    print("Игрок погиб!")
    break
else:
    print(f"Победа! Убито врагов: {kills}, осталось HP: {player_hp}")
```

Здесь есть `for` с `enumerate`, вложенные циклы, `while` с составным условием, `if`, тернарный оператор, `break`, `continue` и `else` у цикла.

11. Упражнения

1. Напиши программу: по числу `n` вывести «чётное» или «нечётное».
2. Выведи числа от 10 до 1, а затем «Пуск!».
3. Посчитай сумму всех чисел от 1 до 100 с помощью `for`.
4. Дан список `[3, -1, 8, -5, 2]`. Создай список только из положительных чисел (сначала циклом, потом генератором).

5. Запрашивай у пользователя пароль, пока он не введёт "dragon".
6. FizzBuzz: для чисел от 1 до 15 выведи Fizz, если делится на 3, Buzz — если на 5, FizzBuzz — если на 3 и на 5, иначе само число.
7. Что выведет этот код и почему?

```
python

for i in range(3):
    if i == 5:
        break
else:
    print("конец")
```

8. Нарисуй «лесенку» из 4 строк с помощью вложенных циклов: *, **, ***, ****.

Ответы

1. Решение:

```
python

n = 7
if n % 2 == 0:
    print("чётное")
else:
    print("нечётное")
```

2. Решение:

```
python

for i in range(10, 0, -1):
    print(i)
print("Пуск!")
```

3. Решение (ответ `5050`, проверка: `sum(range(1, 101))`):

```
python

total = 0
for n in range(1, 101):
    total += n
print(total)    # 5050
```

4. Решение:

```
python

nums = [3, -1, 8, -5, 2]
positive = []
for n in nums:
    if n > 0:
        positive.append(n)

positive = [n for n in nums if n > 0]
print(positive)    # [3, 8, 2]
```

5. Решение:

```
python

while True:
    password = input("Пароль: ")
    if password == "dragon":
        print("Доступ разрешён")
        break
    print("Неверно, попробуй ещё")
```

6. Решение (проверка на 15 должна идти **первой**):

```
python

for n in range(1, 16):
    if n % 15 == 0:
        print("FizzBuzz")
    elif n % 3 == 0:
        print("Fizz")
    elif n % 5 == 0:
        print("Buzz")
    else:
        print(n)
```

7. Выведет **конец**: **i** никогда не равно 5, поэтому **break** не срабатывает, цикл заканчивается сам — и выполняется **else**.

8. Решение:

```
python

for row in range(1, 5):
    line = ""
    for _ in range(row):
        line += "*"
    print(line)
# короче: print("*" * row)
```

Итог

- **if / elif / else** выбирают ветку кода; проверяется сверху вниз, выполняется только первая подходящая.
- **match / case** — удобная замена длинной цепочке **elif** для сравнения одного значения.
- **for** перебирает элементы коллекции; для чисел — **range**, для индексов — **enumerate**, для пар — **zip**.
- **while** повторяет, пока условие истинно; следи, чтобы оно когда-нибудь стало ложным.
- **break** — выйти, **continue** — пропустить итерацию, **else** у цикла — «если не было break».
- **Генераторы списков** — короткий способ построить список циклом.

- Блоки задаются **отступами** и начинаются после **двоеточия**.



Следующая тема — **функции**: `def`, параметры, `return`, область видимости. Они помогут упаковать циклы и условия в переиспользуемые блоки.